

DAA

Lecture 08

Engr. Khurum Iqbal

Properties of asymptotic notation :

- The properties of asymptotic notation are studied in computer science and mathematics because they provide a formal and rigorous way to analyze and describe the efficiency and performance of algorithms and data structures.
- Asymptotic notation allows us to make generalizations about the behavior of functions and how they grow in relation to the input size, without getting into the specifics of constant factors or lower-order terms.

Why Properties ?

The key reasons why we study the properties of asymptotic notation are:

- **Analyzing Algorithm Efficiency:** Asymptotic notation allows us to analyze the efficiency of algorithms by providing a concise and standardized way to express the growth rate of their running time or space complexity. This helps us compare different algorithms and choose the most efficient one for a particular problem.
- **Predicting Algorithm Behavior:** Asymptotic notation provides insights into how an algorithm's performance changes as the input size grows. It helps us understand how the algorithm scales and predict its behavior for large input sizes, which is crucial in real-world scenarios where input sizes can vary greatly.

Why Properties ?

- **Designing Efficient Algorithms:** Asymptotic notation helps in designing efficient algorithms by guiding the choice of data structures, algorithms, and optimization techniques. It allows us to identify and eliminate bottlenecks in the algorithm, and optimize the overall performance.
- **Formalizing Algorithm Analysis:** Asymptotic notation provides a formal and rigorous way to analyze and express the performance of algorithms. It helps in writing clear and concise algorithm analysis proofs and making conclusions based on mathematical principles.

Transitivity

- If $f(n)$ is $O(g(n))$ and $g(n)$ is $O(h(n))$, then $f(n)$ is also $O(h(n))$.
- Example: Let's say we have three functions: $f(n) = 2n$, $g(n) = 3n^2$, and $h(n) = 4n^3$. If we can show that $f(n)$ is $O(g(n))$ and $g(n)$ is $O(h(n))$, then we can conclude that $f(n)$ is also $O(h(n))$.
- $f(n) = 2n$ is $O(g(n))$ because $2n$ grows at most linearly with n , while $g(n) = 3n^2$ grows quadratically with n . $g(n) = 3n^2$ is $O(h(n))$ because $3n^2$ grows at most quadratically with n , while $h(n) = 4n^3$ grows cubically with n .
- So, by transitivity, we can conclude that $f(n) = 2n$ is $O(h(n))$.

Reflexivity

- For any function $f(n)$, $f(n)$ is always $O(f(n))$ and $\Omega(f(n))$.
- Example: Let's consider $f(n) = n^3$. Since $f(n)$ is itself a function of n , it is always $O(f(n))$ and $\Omega(f(n))$.

Symmetry

- If $f(n)$ is $\Theta(g(n))$, then $g(n)$ is also $\Theta(f(n))$.
- Example: Let's say we have $f(n) = n^2$ and $g(n) = n^2$. If we can show that $f(n)$ is $\Theta(g(n))$, then we can conclude that $g(n)$ is also $\Theta(f(n))$.
- $f(n) = n^2$ is $\Theta(g(n))$ because both $f(n)$ and $g(n)$ grow quadratically with n and have similar growth rates.
- So, by symmetry, we can conclude that $g(n) = n^2$ is also $\Theta(f(n))$.

General Property

- If $f(n)$ is $O(g(n))$ then $a \cdot f(n)$ is also $O(g(n))$
- Example
- $f(n) = 3n^2$ is $O(g(n))$ because it grows quadratically with n
- Then
- $a \cdot f(n) = 2(3n^2)$ is $O(g(n))$ because it grows quadratically with n

Addition

- If $f(n)$ is $O(g(n))$ and $h(n)$ is $O(i(n))$, then $f(n) + h(n)$ is $O(\max(g(n), i(n)))$.
- Example: Let's say we have $f(n) = 3n^2$ and $h(n) = 2n^3$. If we can show that $f(n)$ is $O(g(n))$ and $h(n)$ is $O(i(n))$, then we can conclude that $f(n) + h(n)$ is $O(\max(g(n), i(n)))$.
- $f(n) = 3n^2$ is $O(g(n))$ because it grows quadratically with n , and $h(n) = 2n^3$ is $O(i(n))$ because it grows cubically with n .
- So, by addition property, we can conclude that $f(n) + h(n)$ is $O(\max(g(n), i(n))) = O(\max(3n^2, 2n^3))$.

Multiplication

- If $f(n)$ is $O(g(n))$ and $h(n)$ is $O(i(n))$, then $f(n) * h(n)$ is $O(g(n) * i(n))$.
- Example:
- Let's say we have $f(n) = 2n$ and $h(n) = 3n^2$. If we can show that $f(n)$ is $O(g(n))$ and $h(n)$ is $O(i(n))$, then we can conclude that $f(n) * h(n)$ is $O(g(n) * i(n))$.

Asymptotic Notation

- Asymptotic notation is a mathematical tool used to describe the performance characteristics of algorithms and data structures. It allows us to analyze the behavior of these systems as the input size approaches infinity, without worrying about the exact values of constants or lower-order terms. The three most common types of asymptotic notation are:
 - Big O notation
 - Big Omega notation
 - Big Theta notation

Asymptotic Notation

Small O
notation ?

Small Omega
notation ?

Difference between big $O(n)$ and Small $o(n)$ in asymptotic notation

- In asymptotic notation, "Big O " (O) and "Small o " (o) are used to describe the upper bound and strict upper bound, respectively, on the growth rate of a function in relation to the input size (n).
- The main difference between Big O (O) and Small o (o) notations is in the inclusiveness of the upper bound:

Difference between big $O(n)$ and Small $o(n)$ in asymptotic notation

- Big $O(O)$ notation: It provides an upper bound on the growth rate of a function, but it allows for the function to grow at the same rate or slower. In other words, a function $f(n)$ is said to be $O(g(n))$ if there exists positive constants c and n_0 such that $f(n) \leq c * g(n)$ for all $n \geq n_0$.
- Example: If an algorithm has a time complexity of $O(n^2)$, it means that the running time of the algorithm grows no faster than n^2 , but it could be slower, such as n , $\log n$, or constant time.

Difference between big $O(n)$ and Small $o(n)$ in asymptotic notation

- Small $o(o)$ notation: It provides a strict upper bound on the growth rate of a function, meaning that the function must grow strictly slower than the bounding function. In other words, a function $f(n)$ is said to be $o(g(n))$ if for any positive constant c , there exists a positive constant n_0 such that $f(n) < c * g(n)$ for all $n \geq n_0$.
- Example: If an algorithm has a time complexity of $o(n^2)$, it means that the running time of the algorithm grows strictly slower than n^2 , such as n , $\log n$, or constant time.

Difference between big $O(n)$ and Small $o(n)$ in asymptotic notation

- In summary, the difference between Big $O(O)$ and Small $o(o)$ notations lies in the inclusiveness of the upper bound. Big $O(O)$ allows for the function to grow at the same rate or slower, while Small $o(o)$ requires the function to grow strictly slower than the bounding function.
- Both notations are used to describe the growth rate of functions and analyze the efficiency of algorithms in terms of their running time or space complexity.

Thank You.

Questions ??